

ADVANCE PROBLEM PROGRAMMING

SLO 2: Database Processing

1. Write Java Code to Connect to MySQL Using JDBC

JDBC (Java Database Connectivity) is used to connect Java applications with databases such as MySQL.

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/college";
```

```
        String username = "root";
```

```
        String password = "root";
```

```
        try {
```

```
            Connection con = DriverManager.getConnection(
```

```
                url, username, password
```

```
            );
```

```
            System.out.println("Database connected successfully");
```

```
            con.close();
```

```
        } catch (Exception e) {
```

```
            System.out.println("Connection failed");
```

```
            e.printStackTrace();
```

```
        }
```

```
}  
}
```

Output:

Database connected successfully

2. Execute an INSERT and SELECT Statement Using JDBC

```
import java.sql.*;
```

```
public class Main {  
    public static void main(String[] args) {  
  
        String url = "jdbc:mysql://localhost:3306/college";  
        String username = "root";  
        String password = "root";  
  
        try {  
            Connection con = DriverManager.getConnection(  
                url, username, password  
            );  
  
            // INSERT statement  
            String insertQuery =  
                "INSERT INTO student (id, name) VALUES (101, 'Rahul')";  
  
            Statement stmt = con.createStatement();  
            stmt.executeUpdate(insertQuery);  
        }  
    }  
}
```

```
System.out.println("Record inserted successfully");

// SELECT statement
String selectQuery = "SELECT * FROM student";
ResultSet rs = stmt.executeQuery(selectQuery);

System.out.println("Student Records:");

while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name")
    );
}

con.close();

} catch (Exception e) {
    e.printStackTrace();
}
}
```

Output:

Record inserted successfully

Student Records:

The `executeUpdate()` method is used for the INSERT operation, while `executeQuery()` is used for the SELECT operation.

3. Explain the Purpose of Prepared Statements in Java

A **PreparedStatement** is used to execute SQL statements with parameters. It provides better security, performance, and readability compared with building SQL queries using string concatenation.

```
import java.sql.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/college";
```

```
        String username = "root";
```

```
        String password = "root";
```

```
        try {
```

```
            Connection con = DriverManager.getConnection(
```

```
                url, username, password
```

```
            );
```

```
            String sql =
```

```
                "INSERT INTO student (id, name) VALUES (?, ?)";
```

```
            PreparedStatement ps = con.prepareStatement(sql);
```

```
ps.setInt(1, 102);  
ps.setString(2, "Arun");  
  
ps.executeUpdate();  
  
System.out.println("Record inserted successfully");  
  
con.close();  
  
} catch (Exception e) {  
    e.printStackTrace();  
}  
}  
}
```

Advantages of PreparedStatement:

1. **Prevents SQL Injection** by separating SQL code from user-supplied values.
2. **Improves performance** when the same SQL statement is executed multiple times.
3. **Makes code easier to read and maintain.**
4. **Handles different data types** using methods such as `setInt()`, `setString()`, and `setDouble()`.